Code walkthrough

# flag_register.v

## Flag Register

| | |
|---|---|
| What it does | Remembers the Zero and Carry flags |
| Where it works | EX stage |
| Size | 34 lines, explained in 3 small parts |
| Modules in this file | flag_register |
| Calls (uses) | nothing (no other modules) |
| Called by | cpu_top.v - instance FLAGS |

The ALU produces Zero and Carry for every calculation, but those are only wires. The flag register saves them at the clock edge so a later instruction (JZ) can use them.

## Map of the file

| Part | Lines | What this part is |
|---|---|---|
| 1 | 1-13 | Header comment |
| 2 | 14-23 | Ports |
| 3 | 24-34 | Clocked behaviour |

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

## Part 1 · Header comment (lines 1-13)

**flag_register.v lines 1-13**

```
 1  // ======================================================================
 2  // FILE: flag_register.v
 3  // Saves the Zero and Carry flags so a later instruction (JZ) can use
 4  // them, and so they can be observed in the waveform.
 5  //   zero  : last arithmetic/logic result was 0000
 6  //   carry : carry-out from ADD, or NO-BORROW from SUB
 7  // They have separate enables because AND/OR/XOR/NOT change only Zero,
 8  // while ADD/SUB change both Zero and Carry.
 9  //
10  // CONNECTIONS
11  //   Calls       : nothing (no other modules)
12  //   Called by   : cpu_top.v - instance FLAGS
13  // ======================================================================
```

- Explains both flags and why each has its own write enable.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

## Part 2 · Ports (lines 14-23)

**flag_register.v lines 14-23**

```
14  module flag_register (
15      input      clk,
16      input      rst,
17      input      zero_we,     // 1 = save new zero
18      input      carry_we,    // 1 = save new carry
19      input      zero_in,     // from the ALU
20      input      carry_in,    // from the ALU
21      output reg zero_flag,   // saved zero
22      output reg carry_flag   // saved carry
23  );
```

- `zero_we`, `carry_we`: separate write enables.
- `zero_in`, `carry_in`: new values from the ALU.
- `zero_flag`, `carry_flag`: the saved values.

## Part 3 · Clocked behaviour (lines 24-34)

**flag_register.v lines 24-34**

```
24      always @(posedge clk) begin
25          if (rst) begin
26              zero_flag  <= 1'b0;
27              carry_flag <= 1'b0;
28          end
29          else begin
30              if (zero_we)  zero_flag  <= zero_in;
31              if (carry_we) carry_flag <= carry_in;
32          end
33      end
34  endmodule
```

- Reset clears both flags.
- Otherwise each flag is updated only when its own enable is 1. ADD and SUB turn on both enables; AND, OR, XOR and NOT turn on only `zero_we`, so Carry keeps its old value; LDI, MOV, LOAD, STORE and jumps turn on neither.

**In short:** Two separately-enabled 1-bit registers. **Example:** ADD 15 + 1 gives zero_in = 1, carry_in = 1 with both enables on, so both flags become 1. A following OR has zero_we = 1 but carry_we = 0, so Zero is updated and Carry stays 1.